

Deep Learning: CNN

Vladimir I N Bykov

Nyckelbegrepp:

Objektigenkänning: Processen att identifiera och lokalisera objekt i en bild, till exempel att känna igen en bil, ett ansikte eller en skylt.

Bildklassificering: Att tilldela en bild en viss kategori baserat på innehållet, till exempel att avgöra om en bild innehåller en katt eller hund.

Bildsegmentering: Att dela upp en bild i olika regioner eller segment, till exempel att segmentera en medicinsk bild för att identifiera olika organ eller vävnader.

Varför är neurala nätverk lämpliga för bildanalys?

Automatisk funktionsextraktion: Neurala nätverk lär sig automatiskt att extrahera visuella funktioner som kanter och texturer, och bygger dessa till högre koncept som objekt och scener.

Skalbarhet: Neurala nätverk kan hantera stora mängder data, vilket är viktigt för bildanalys där stora dataset ofta används.

Robusthet: De kan hantera variationer i bilder, såsom förändringar i ljus, perspektiv, och bakgrund, mycket bättre än traditionella metoder.

Översikt av Konvolutionella Neurala Nätverk (CNN)

Konvolutionella Neurala Nätverk (CNNs) är en typ av djupa neurala nätverk som är särskilt utformade för att hantera bilddata. De utnyttjar två viktiga mekanismer: konvolution och pooling, vilket gör dem exceptionellt bra på att förstå och analysera visuella mönster i bilder. CNNs har revolutionerat bildanalys och används i allt från ansiktsigenkänning till medicinsk bildanalys.

Konvolution (Convolution): Detta är en operation där ett filter (eller kernel) glider över bilden och skapar en feature map genom att detektera mönster som kanter, linjer eller färgvariationer. Filtret hjälper nätverket att fokusera på lokala mönster i bilden.

Pooling: Efter konvolution utförs ofta en pooling-operation (som max-pooling) för att minska dimensionerna på feature maps. Detta steg hjälper till att minska beräkningskostnaden, kontrollera överanpassning och göra nätverket robustare mot små förändringar i bilden (som små förskjutningar eller brus).

Fullständiga Lager (Fully Connected Layers): Efter att ha bearbetat bilden genom flera konvolutionella och pooling-lager, används fullständiga lager (fully connected layers) för att fatta beslut baserat på de extraherade funktionerna. Dessa lager liknar traditionella neurala nätverk och ansvarar för den slutliga klassificeringen av bilden.

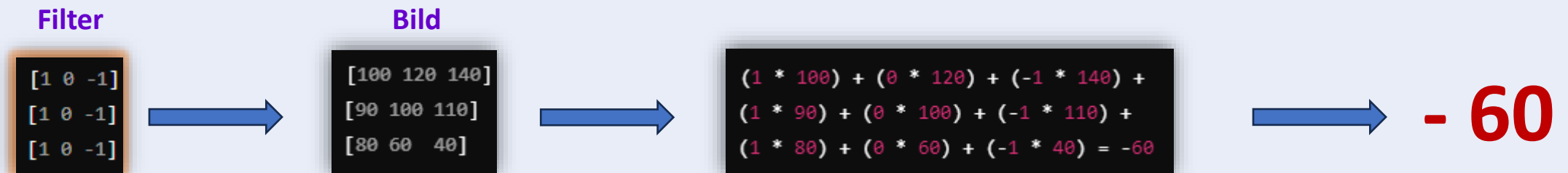
Convolution (CNN)

När vi pratar om konvolution i bildanalys och CNNs sker följande steg vid varje position:

Bildens pixelvärden: Varje bild består av ett rutnät av pixelvärden. Om det är en gråskalig bild är varje pixel representerad av ett numeriskt värde mellan 0 och 255 (där 0 är svart och 255 är vitt). För färgbilder finns det flera kanaler (Röd, Grön, Blå) där varje kanal har sitt eget rutnät av pixelvärden.

Filter (eller kernel): Ett filter, även kallat en kernel, är en mindre matris av tal, t.ex. 3x3 eller 5x5, som används för att extrahera specifika mönster från bilden, som kanter eller texturer.

Multiplikation vid varje position: Filtrets storlek bestämmer hur många pixlar som behandlas samtidigt. Om vi har ett 3x3 filter, tas 3x3 pixelvärden från bilden. Varje värde i detta 3x3 bildområde multipliceras med motsvarande värde i filtret.



Summan lagras: Denna summa (i vårt exempel -60) lagras som ett nytt pixelvärde i en ny matris (som representerar den bearbetade bilden). Detta processas för alla pixelpositioner i bilden, där filtret "glider" över bilden (en så kallad stridning).

Konvolutionen hjälper datorn att upptäcka mönster som kanter och konturer, och detta görs genom att förstärka vissa funktioner i bilden och undertrycka andra beroende på hur filtret är uppbyggt. Konvolutionen är inte exakt samma som PCA eller UMAP, men båda förenklar datan och lyfter fram de viktigaste egenskaperna, fast på olika sätt och med olika mål.

Manuell (enkel) feature-extraktion med filter

Vi designar filter för att hitta kanter → bygger fönster av kanterna

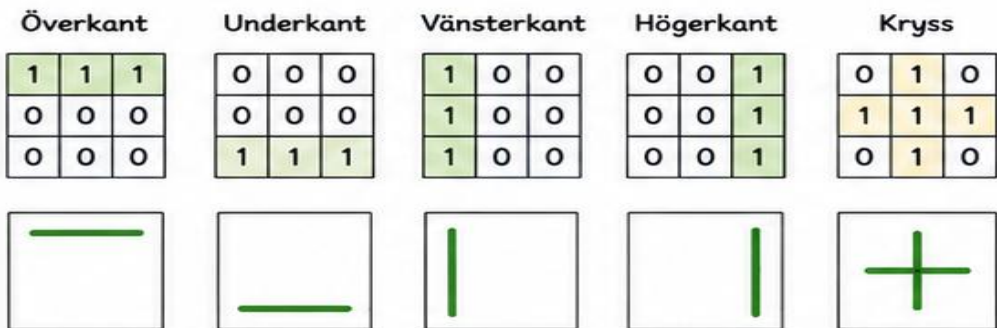
1. Indata: Bild



Gråskalebild
(1 kanal)

2. Manuellt designade filter

Vi skapar filter som letar efter enkla mönster



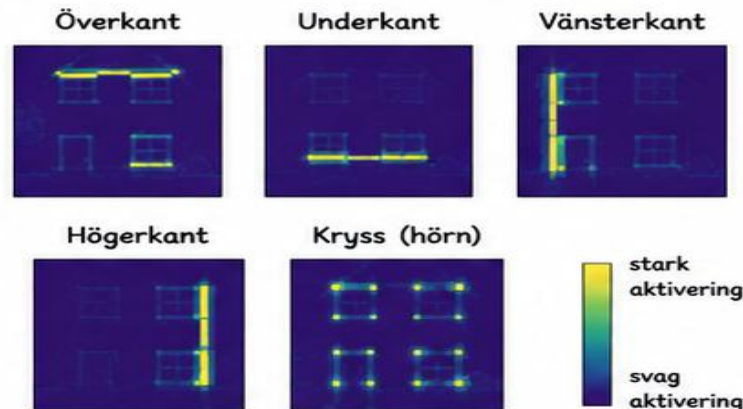
1 = tittar efter ljus (kant)

0 = ignorerar

(filterstorlek: 3x3)

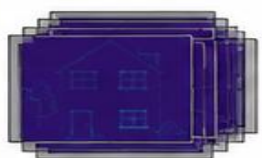
3. Konvolution (filter sveps över bilden)

Varje filter skapas en feature map

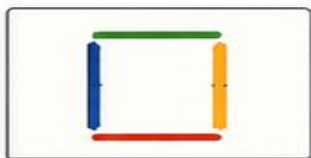


4. Kombinera kanter till former

Genom att titta på flera feature maps samtidigt kan vi hitta rektanglar (fönster, dörrar)



Alla kant-maps
(t.ex. över-, under-,
vänster-, högerkant)

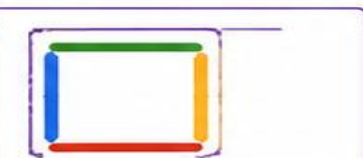
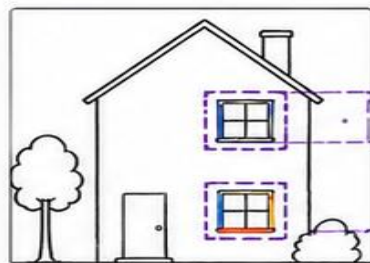


Rektanglar hittas där
kanter bildar slutna former

— överkant — vänsterkant
— underkant — högerkant

5. Fokusera på fönster

Ett fönster = uppfyller alla fyra kanter



✓ överkant finns
✓ underkant finns
✓ vänsterkant finns
✓ högerkant finns

→ Det är ett fönster!

6. Resultat

Fönster lokaliseras



(klass: fönster + position + score)

Fönster 1
x: 72
y: 38
w: 26
h: 22
score: 0.92

Fönster 2
x: 72
y: 74
w: 26
h: 22
score: 0.90



Så fungerar det (manuellt):

1. Designa enkla filter som letar efter kanter (horisontella, vertikala, kors).
2. Applicera (konvolvera) filtren på bilden → få feature maps.
3. Kombinera kanterna för att hitta former (rektanglar).
4. Om alla fyra kanter finns → vi har ett fönster.

Detta är grunden.
I verkliga CNN lär sig nätverket
själv vilka filter som är bäst!



0	0	0	0	0	0	0	0
0	1	2	3	1	3	5	0
0	2	2	5	4	2	5	0
0	0	6	9	6	2	2	0
0	2	0	1	9	4	0	0
0	5	5	4	6	7	6	0
0	6	1	3	7	1	5	0
0	0	0	0	0	0	0	0

*

1	0	-1
1	0	-1
1	0	-1

=

-4	-5	-1	3	-5	5
-10	-14	-1	10	-1	7
-8	-11	-11	7	12	8
11	-7	-10	1	13	13
-6	5	-16	-4	10	12
-6	4	-7	-1	2	8

Padding

När ett filter (eller kernel) appliceras på en bild under en konvolution, centreras filtret på varje pixel som det bearbetar. Detta innebär att varje pixel i bilden (eller feature mapen) behandlas som "mitten" av en region som filtret analyserar. Filtret använder de omgivande pixlarna runt den centrala pixeln för att beräkna en ny pixelvärde i utdata.

Problemet uppstår när filtret når kanterna på bilden. När filtret försöker centreras på en kantpixel, finns det inte tillräckligt med pixlar runt om för att fylla alla positioner i filtret.

Filtret måste centreras på varje pixel för att:

Jämnt och konsekvent bearbeta hela bilden: Genom att centrera filtret på varje pixel får vi en symmetrisk och konsekvent konvolutionsoperation, där varje pixel behandlas likadant.

Bearbeta omgivande information: En av styrkorna med konvolutionella nätverk är deras förmåga att analysera lokal kontext. Genom att centrera filtret på varje pixel kan nätverket fånga information om både pixeln själv och dess omgivning.

Vad händer utan centrering?

Om filtret inte centreras på pixlar, skulle vi inte kunna använda information från omgivande pixlar på ett enhetligt sätt, och vi skulle missa en stor del av konvolutionsoperationens syfte – att lära sig lokala mönster och strukturer från en bild.

Hur padding hjälper:

När vi använder padding (vanligtvis genom att lägga till nollor runt kanten av bilden) kan filtret fortfarande centreras på kantpixlarna, eftersom nollorna fungerar som extra pixlar. Detta gör att filtret kan fortsätta att centreras korrekt, även vid kanterna, utan att "ramla av" bilden.

Utan padding blir det en gradvis förlust av information med varje konvolutionslager, och feature maps (utdata) blir mindre och mindre i storlek för varje lager. Detta beror på att utan padding kan filtret inte appliceras fullt ut vid kanterna av bilden, vilket gör att antalet pixlar som bearbetas minskar vid varje steg.

Padding: options

```
# Bygg CNN-modellen
model = models.Sequential()

# Första konvolutionella lagret
model.add(layers.Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(48, 48, 1)))
model.add(layers.MaxPooling2D((2, 2)))

# Andra konvolutionella lagret
model.add(layers.Conv2D(64, (3, 3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D((2, 2)))

# Tredje konvolutionella lagret
model.add(layers.Conv2D(128, (3, 3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D((2, 2)))

# Fjärde konvolutionella lagret
model.add(layers.Conv2D(256, (3, 3), activation='relu', padding='same'))
model.add(layers.MaxPooling2D((2, 2)))

# Flatten-lager
model.add(layers.Flatten())

# Fullt anslutna lager
model.add(layers.Dense(128, activation='relu'))
model.add(layers.Dropout(0.5)) # Dropout för att minska överanpassning

# Lägg till ytterligare ett fullt anslutet lager med Dropout
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dropout(0.3))

# Utgångslagret
model.add(layers.Dense(7, activation='softmax'))

# Kompilera modellen
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

valid padding är default: ingen padding
same padding: padding med nollor runt om bilden

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel		
0	-1	0
-1	5	-1
0	-1	0

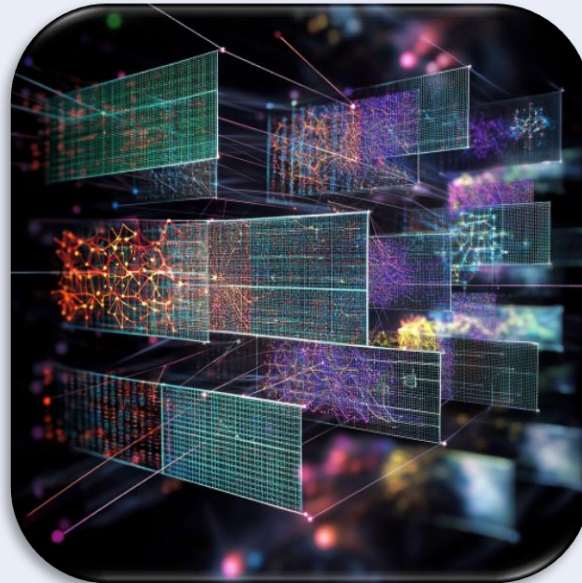
114	328	-26	470	158
53	266	-61	-30	344
403	116	-47	295	244
108	-135	256	-128	344
314	346	279		

Feature maps

När filtret (kernel) glider över bilden under **konvolutionsoperationen** genereras nya representationer av bilden kallade **feature maps**. Dessa är i praktiken nya bilder, där varje pixel indikerar hur starkt filtret reagerade på en specifik plats i originalbilden.

Om filtret exempelvis letar efter kanter, kommer feature map tydligt visa var dessa kanter finns i bilden.

Flera feature maps: CNN använder flera olika filter för att hitta olika egenskaper i bilden. Varje filter skapar sin egen feature map. Tillsammans ger dessa feature maps en samlad representation av olika egenskaper i bilden, som hjälper nätverket att förstå och tolka bilden.



Pooling

Max-pooling (vanligast förekommande):

- Bilden delas upp i små rutor (exempelvis 2x2 pixlar).
- Från varje ruta behåller nätverket endast det starkaste värdet (maxvärdet).
- Exempel: Om ett område innehåller en kant eller en annan tydlig struktur, kommer max-pooling att behålla denna struktur och ignorera mindre viktiga delar.

Exempel:

Tänk dig en bild av en katt.

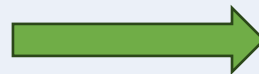
När vi använder max-pooling kan även små förskjutningar av katten i bilden ignoreras eftersom pooling fokuserar på de starkaste kännetecknen (t.ex. kanter, öron, ögon), vilket gör nätverket mer stabilt och precist även vid varierande input.

I sammanfattning hjälper pooling oss att:

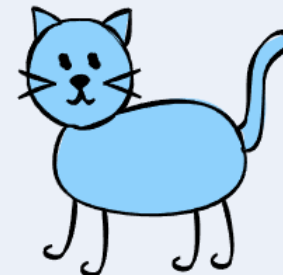
- Minska mängden data som måste bearbetas.
- Förbättra nätverkets effektivitet och minska risken för överanpassning.
- Göra nätverket mer motståndskraftigt mot små förändringar i bilden.



convolution



max pooling



Fullständiga Lager (Fully Connected Layers)

Fullständiga lager, eller Fully Connected Layers (FC-lager), är de sista lagren i ett konvolutionellt neuralt nätverk (CNN). De fungerar på samma sätt som lagren i traditionella neurala nätverk och är avgörande för nätverkets slutliga beslut.

Efter konvolution och pooling:

När en bild har gått igenom flera konvolutionella och pooling-lager har nätverket extraherat många olika visuella egenskaper (så kallade features). Dessa features sammanställs i en lång vektor (ett "plattat" lager) som innehåller all viktig information som nätverket identifierat i bilden.

Ett fullständigt lager tar den sammanställda informationen och bearbetar den för att fatta ett slutligt beslut. Varje neuron i ett fullständigt lager är kopplat till samtliga features från det föregående lagret. På så sätt använder nätverket all tillgänglig information för att avgöra vilken kategori eller klass bilden tillhör.

Fullständiga Lager (Fully Connected Layers)

Hur det fungerar i praktiken:

- Anta att vi tränar ett nätverk för att identifiera olika djur på bilder, exempelvis katter och hundar.
- De tidigare konvolutionella och pooling-lagren identifierar och extraherar visuella detaljer som öron, ögon, pälsstruktur eller kroppsform.
- De fullständiga lagren använder sedan dessa extraherade detaljer för att fatta ett beslut: "Är detta en bild av en katt eller en hund?"

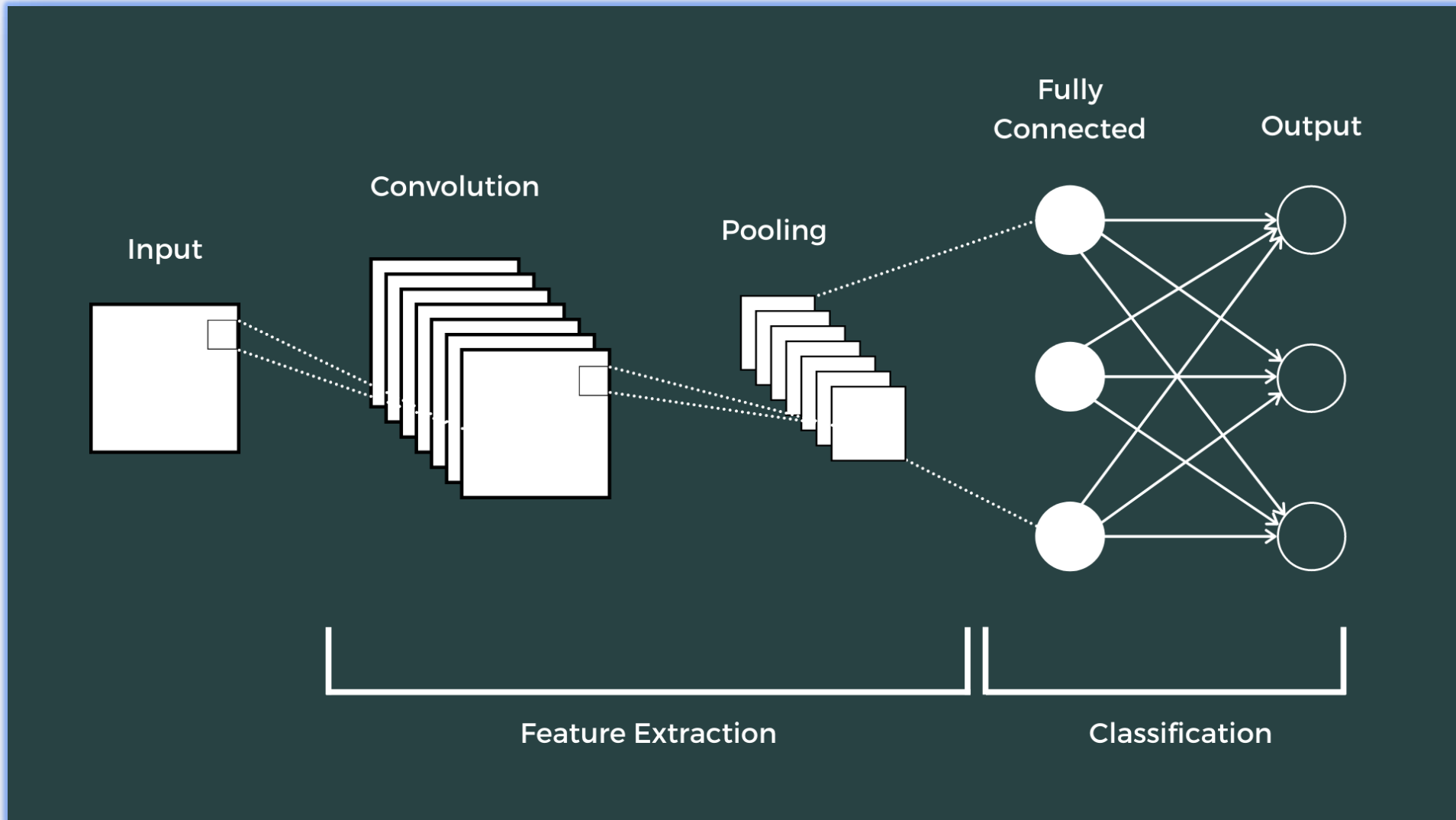
Slutlig klassificering:

Fullständiga lager ger det slutliga svaret. Dessa lager avgör om bilden föreställer exempelvis en katt, en hund eller något annat objekt som nätverket tränats att känna igen.

Sammanfattning:

Fullständiga lager fungerar som beslutsfattare i ett CNN. De tar all viktig extraherad information från tidigare lager, bearbetar den, och genererar slutligen nätverkets bedömning av bilden.

CNN



Fördelar med CNN för Bildanalys

Automatiserad Funktionsextraktion:

CNNs behöver inte att manuellt designa och extrahera egenskaper från bilderna, vilket gör dem överlägsna traditionella metoder.

De lär sig automatiskt från data genom att identifiera mönster, former, och detaljer i bilden. Detta innebär att CNNs kan hitta både enkla och komplexa egenskaper (som kanter och former) genom flera lager av nätverket.

Translationsinvarians:

Translationsinvarians innebär att CNNs kan känna igen ett objekt oavsett var det finns i bilden. Det spelar ingen roll om ett objekt, som ett ansikte, är centrerat eller placerat i ett hörn – CNN kan fortfarande känna igen det.

Detta sker tack vare pooling-operationer och delade vikter, som gör nätverket robust mot förskjutningar och små förändringar i bildens position.

Effektiv Beräkning:

CNNs är mycket mer beräkningsmässigt effektiva jämfört med vanliga neurala nätverk för bildanalys.

Genom att använda konvolutioner med delade vikter, där samma filter används för olika delar av bilden, minskas antalet parametrar drastiskt. Detta sparar både minne och beräkningstid, samtidigt som nätverket kan tränas på större datamängder och djupare nätverk.

Tack för er tid